0253. Meeting Rooms II

Medium | Category: Intervals | LeetCode: https://leetcode.com/problems/meeting

Problem

- Return the minimum number of conference rooms required for all meetings.

Intuition

- The answer is the maximum number of meetings in progress at any moment.
- Sweep line: sort starts and ends separately; walk through the starts, and whenever the earliest end is <= the current start, a room frees up.

Brute Force

- For every time point count overlapping meetings: O(n T).

Optimal Approach

- Starts sorted, ends sorted; rooms = 0, e = 0.
- For each start s: if s >= ends[e], e++ (reuse a room); else rooms++.

Worked Example

```
[[0,30],[5,10],[15,20]].

starts 0 5 15 | ends 10 20 30
s=0  : 0 < 10   -> rooms 1
s=5  : 5 < 10   -> rooms 2
s=15 : 15 >= 10 -> reuse, e=1
answer 2
```

Java Solution

```java
public class MeetingRoomsII {
    public int minMeetingRooms(int[][] intervals) {
        int n = intervals.length;
        int[] starts = new int[n], ends = new int[n];
        for (int i = 0; i < n; i++) { starts[i] = intervals[i][0]; ends[i] = intervals[i][1]; }
        Arrays.sort(starts);
        Arrays.sort(ends);
        int rooms = 0, e = 0;
        for (int s : starts) {
            if (s >= ends[e]) e++;
            else rooms++;
        }
        return rooms;
    }
}
```

Complexity

- Time: O(n log n). Space: O(n).

Pitfalls

- A meeting ending at t frees the room for one starting at t (>=).
- A min-heap of end times is an equivalent and popular alternative.